TUGAS II
IF2211 Strategi Algoritma

Pemanfaatan Algoritma BFS dan DFS dalam Mekanisme Penelusuran CSS pada pohon Document Object Model

Tanggal Pengumpulan: Sabtu, 25 April 2026

Anggota:

Ega Luthfi Rais	13524115
Ahmad Fauzan Putra	13524141
Muh. Hartawan Haidir	13524147

INSTITUT TEKNOLOGI BANDUNG

2025/2026

DAFTAR ISI

DAFTAR ISI 1
BAB I DESKRIPSI TUGAS 2
BAB II LANDASAN TEORI 4
BAB III APLIKASI BFS DAN DFS 7
BAB IV IMPLEMENTASI DAN PENGUJIAN 8
BAB V KESIMPULAN DAN SARAN 16
LAMPIRAN 17
DAFTAR PUSTAKA 19

BAB I DESKRIPSI TUGAS

Pada tugas yang telah diberikan, mahasiswa akan membuat aplikasi traversal pohon HTML (DOM Tree) yang menggunakan algoritma Breadth First Search (BFS) dan Depth First Search (DFS) untuk melakukan pencarian elemen berdasarkan CSS selector pada struktur DOM.

- Mahasiswa akan membuat aplikasi berbasis web yang terdiri dari frontend dengan menggunakan bahasa yang bebas dan backend dengan penggunaan bahasa yang telah ditentukan, yaitu `Go, C#, Rust, atau Typescript` yang mengimplementasikan **algoritma BFS dan DFS** dalam melakukan penelusuran CSS pada pohon DOM
- Aplikasi dapat melakukan scraping HTML dari sebuah website melalui URL yang diberikan oleh pengguna
- Secara umum, berikut adalah input yang diharapkan
    - URL website atau memasuki teks HTML sendiri
    - Pilihan algoritma traversal (BFS / DFS_
    - CSS Selector
    - Jumlah hasil yang ingin ditampilkan :
        - Top n kemunculan
        - Semua kemunculan
- HTML yang diperoleh kemudian diparsing menjadi struktur data pohon (DOM Tree).
- Output yang diharapkan adalah sebagai berikut:
    - Aplikasi dapat menampilkan visualisasi struktur pohon DOM, disertai dengan keterangan kedalaman maksimum tree
    - Aplikasi dapat menampilkan highlight pada jalur yang di-traversal oleh algoritma penelusuran yang dipilih untuk menandakan elemen-elemen yang terpengaruh.
    - Aplikasi dapat menampilkan waktu pencarian serta banyak node yang dikunjungi.
    - Setelah melakukan penelusuran, aplikasi menyimpan sebuah traversal log yang memiliki informasi tahapan penelusuran yang dilakukan.
- Repository bagian frontend dan backend boleh digabung / dipisah

Selain membuat aplikasi yang telah dijelaskan, mahasiswa juga dapat membuat bagian berikut sebagai bonus pada tugas :

- **Membuat video** : video tentang aplikasi BFS dan DFS pada penelusuran pohon DOM kemudian mengunggahnya di Youtube. Video dibuat harus memiliki audio dan menampilkan wajah dari setiap anggota kelompok. Untuk contoh video tubes stima tahun-tahun sebelumnya dapat dilihat di Youtube dengan kata kunci "Tubes Stima", "strategi algoritma", "Tugas besar stima", dll. Semakin menarik video, maka semakin banyak poin yang diberikan.
- **Mmebuat deploy** : Aplikasi web yang telah dibuat perlu di deploy menggunakan Microsoft Azure Virtual Machine (VM) sehingga aplikasi dapat diakses secara publik melalui internet. Sebagai mahasiswa, terdapat kredit gratis sebesar $100 dari Microsoft untuk dapat digunakan hanya dengan menggunakan email kampus ITB. Karena layanan ini menggunakan sistem pembayaran berbasis penggunaan (pay-as-you-go), penggunaan Virtual Machine akan mengurangi kredit yang dimiliki. Oleh karena itu, disarankan untuk mematikan (stop/deallocate) Virtual Machine ketika tidak digunakan agar kredit Azure tidak cepat habis.
- **Docker** : Aplikasi dijalankan menggunakan Docker baik untuk frontend maupun backend.
- **Animasi** : Menampilkan animasi penelusuran pohon DOM untuk segala pencarian.
- **Multithreading** : Mengimplementasikan multithreading pada algoritma BFS dan/atau DFS untuk melakukan traversal pohon DOM secara paralel. Proses traversal dibagi ke beberapa thread/worker sehingga beberapa node dapat diproses secara bersamaan
- **LCA Binary Lifting** : Mengimplementasikan algoritma Lowest Common Ancestor (LCA) dengan teknik Binary Lifting pada pohon DOM yang dihasilkan dari proses parsing HTML. LCA digunakan untuk menentukan node leluhur terendah yang sama dari dua node dalam sebuah pohon. Dalam konteks DOM, fitur ini dapat digunakan untuk mengetahui elemen HTML terdekat yang menjadi parent bersama dari dua elemen tertentu. Penjelasan lebih lanjut dapat dilihat pada link berikut.

BAB II LANDASAN TEORI

1. Document Object Model (DOM)

Document Object Model (DOM) merupakan representasi terstruktur dari dokumen HTML dalam bentuk tree yang memungkinkan setiap elemen pada halaman web diakses, dimodifikasi, dan dimanipulasi oleh program. Struktur ini merepresentasikan hubungan antar elemen HTML seperti parent, child, dan sibling sehingga memudahkan pengolahan dokumen secara terprogram. Struktur sebuah file HTML pada umumnya meliputi elemen root <html> yang berisi dua bagian utama yaitu <head> dan <body>. Bagian <head> berisi metadata seperti judul halaman, link stylesheet, dan script, sedangkan <body> berisi konten utama halaman seperti teks, gambar, tombol, dan elemen HTML lainnya yang ditampilkan kepada pengguna. Sebuah website menampilkan sebuah DOM melalui sebuah file bernama Cascading Style Sheets (CSS) yang bertujuan mendeklarasikan visualisasi elemen-elemen pada pohon. Deklarasi visualisasi ditempelkan pada elemen-elemen yang berkaitan melalui CSS Selector.

2. CSS Selector

CSS Selector adalah mekanisme pada CSS untuk memilih elemen HTML tertentu pada struktur DOM agar dapat diberikan aturan style. Jenis selector CSS umum, yaitu :

1. Selektor Universal (*) : Mencocokkan semua elemen dalam dokumen
2. Pemilih Elemen/Tag : Memilih elemen berdasarkan nama tagnya (misalnya, `document.querySelector('p')` memilih paragraf pertama
3. Pemilih ID (#): Menargetkan elemen unik dengan ID tertentu (misalnya, `#header`)
4. Pemilih Kelas (.): Memilih elemen dengan atribut kelas tertentu (misalnya, `.btn-primary`)
5. Pemilih atribut ([]): Menargetkan elemen berdasarkan atribut atau nilainya (misalnya, `input[type="text"]`)
6. Kombinator: Mendefinisikan hubungan antar elemen, seperti Keturunan (spasi), Anak (>), atau Saudara Seperempat (+).
7. Kelas semu (:) : Memilih elemen dalam keadaan tertentu (misalnya, `:hover`, `:first-child`, atau `:checked`)

Keunggulan CSS Selector dalam DOM adalah

1. Fleksibilitas: Tidak seperti metode lama seperti `getElementById()`, metode querySelector dapat menangani kueri kompleks yang melibatkan kelas, atribut, dan hierarki secara bersamaan
2. Efisiensi: Selector CSS umumnya lebih cepat dan lebih mudah dibaca daripada alternatif seperti XPath untuk sebagian besar tugas penelusuran DOM.
3. Kemudahan penggunaan: Pengembang dapat menggunakan sintaks yang sama untuk penataan gaya (CSS) dan skrip (JavaScript/DOM)

3. Tags

Tag pada HTML merupakan penanda (markup) yang digunakan untuk mendefinisikan struktur dan jenis elemen dalam sebuah dokumen HTML. Tag memberi tahu browser bagaimana suatu bagian konten harus ditafsirkan dan ditampilkan pada halaman web. Secara umum, tag HTML ditulis menggunakan tanda kurung sudut (< >). Sebagian besar elemen HTML memiliki pasangan tag pembuka dan tag penutup. Tag pembuka menandai awal suatu elemen, sedangkan tag penutup menandai akhir elemen tersebut. Tag penutup ditulis dengan menambahkan tanda garis miring / sebelum nama tag. Terdapat beberapa jenis tag HTML yang self-closing.

4. HTML Parsing

HTML parsing adalah proses membaca dokumen HTML dan mengubahnya menjadi struktur data pohon (tree) yang merepresentasikan hubungan antar elemen HTML. Pada proses ini, parses memproses dokumen HTML secara berurutan, mengenali tag pembuka dan tag menutup, kemudian menyusunnya ke dalam struktur hierarki. Setiap tag HTML direpresentasikan sebagai node pada pohon. Tag yang berada di dalam tag lain akan menjadi child node, sedangkan tag yang membungkusnya menjadi parent node. Node paling atas pada struktur ini disebut root, yang umumnya merupakan elemen . Dari node root tersebut, elemen-elemen lain seperti dan akan menjadi cabang yang membentuk struktur pohon DOM. Hasil dari proses parsing adalah DOM Tree, yaitu representasi pohon dari dokumen HTML yang memungkinkan elemen-elemen pada halaman web ditelusuri, diakses, dan dimanipulasi secara terstruktur. Struktur pohon ini kemudian dapat digunakan dalam proses penelusuran elemen, misalnya dengan menggunakan algoritma Breadth First Search (BFS) atau Depth First Search (DFS).

5. **Breadth-First Search (BFS)**

   Breadth-First Search adalah algoritma pencarian graf atau pohon yang menjelajahi simpul secara melebar, mendahulukan tetangga terdekat (level saat ini) sebelum pindah ke level yang lebih dalam. BFS menggunakan struktur data queue (antrian FIFO) untuk melacak simpul yang akan dikunjungi. Sangat efektif untuk menemukan jalur terpendek pada graf yang tidak berbobot.

6. **Depth-First Search (DFS)**

   Depth-First Search (DFS) adalah algoritma penelusuran atau pencarian yang digunakan untuk menjelajahi struktur data graf (graph) atau pohon (tree) dengan cara kerja yaitu menelusuri satu cabang hingga ke tingkat terdalam sebelum melakukan backtracking (kembali ke cabang sebelumnya).

BAB III APLIKASI BFS DAN DFS

1. **Penerapan BFS pada Pohon DOM**

   BFS menelusuri pohon DOM secara level-order menggunakan struktur antrian. Penelusuran dimulai dari node root yang dimasukkan ke antrian. Pada setiap iterasi, satu node dikeluarkan dari antrian, diperiksa terhadap CSS Selector, dan ditambahkan ke daftar hasil jika memenuhi. Seluruh child dari node tersebut kemudian dimasukkan ke belakang antrian. Proses berhenti ketika antrian kosong atau jumlah hasil mencapai batas yang ditentukan pengguna.

2. **Penerapan DFS pada Pohon DOM**

   DFS menelusuri pohon DOM secara pre-order menggunakan rekursi. Dari node root, fungsi rekursif memeriksa node terhadap CSS Selector, lalu memanggil dirinya untuk setiap child secara berurutan dari kiri ke kanan. Algoritma menelusuri satu cabang sampai ke daun sebelum melakukan backtracking ke cabang berikutnya, sehingga urutan penemuan elemen mengikuti urutan kemunculannya pada dokumen HTML.

3. **Perbandingan BFS dan DFS**

   BFS dan DFS menghasilkan kumpulan elemen yang sama untuk CSS Selector yang sama, namun berbeda dalam urutan penemuan dan kebutuhan memori. BFS menemukan elemen yang lebih dekat dengan root terlebih dahulu, sedangkan DFS mengikuti urutan kemunculan elemen pada dokumen HTML. Dari sisi memori, BFS bergantung pada lebar pohon karena menyimpan seluruh node pada level yang sama, sedangkan DFS bergantung pada tinggi pohon karena hanya menyimpan jalur dari root ke node saat ini.

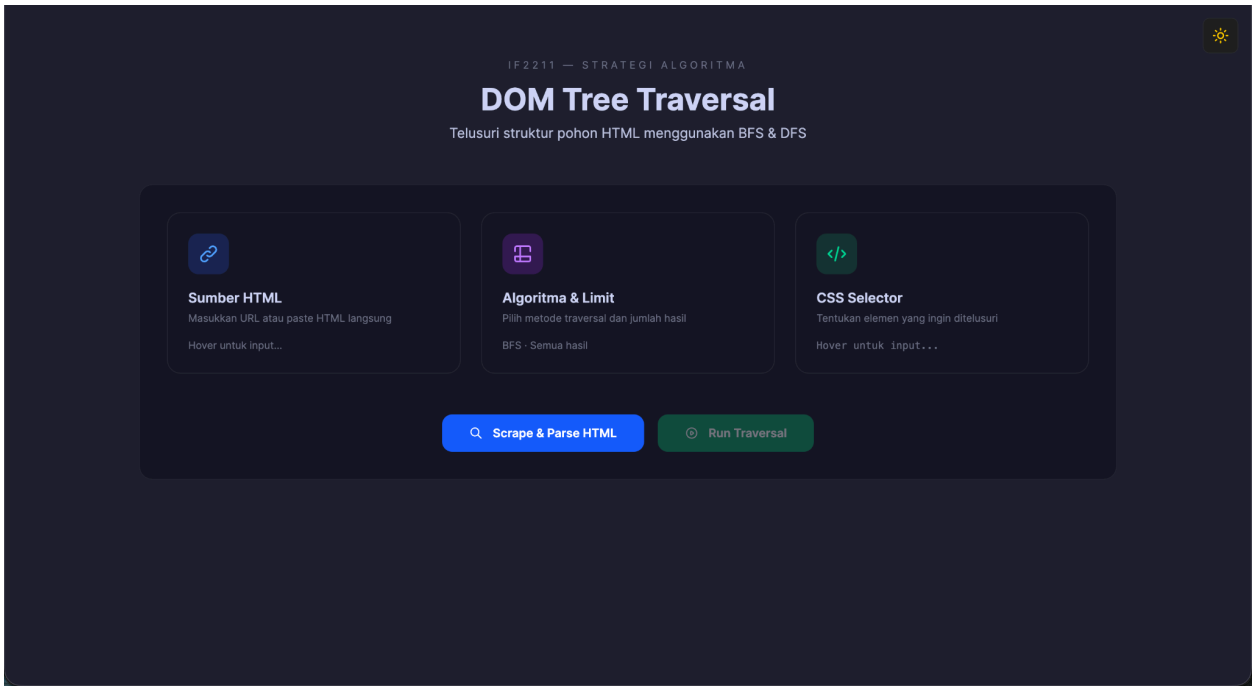
BAB IV IMPLEMENTASI DAN PENGUJIAN

Arsitektur Aplikasi

Aplikasi terdiri dari backend yang melakukan scraping serta traversal dan frontend yang menampilkan input dan visualisasi pohon DOM. Keduanya berkomunikasi melalui API, dijalankan dalam container Docker, dan juga di deploy melalui VM MS Azure sebagai bagian dari bonus.

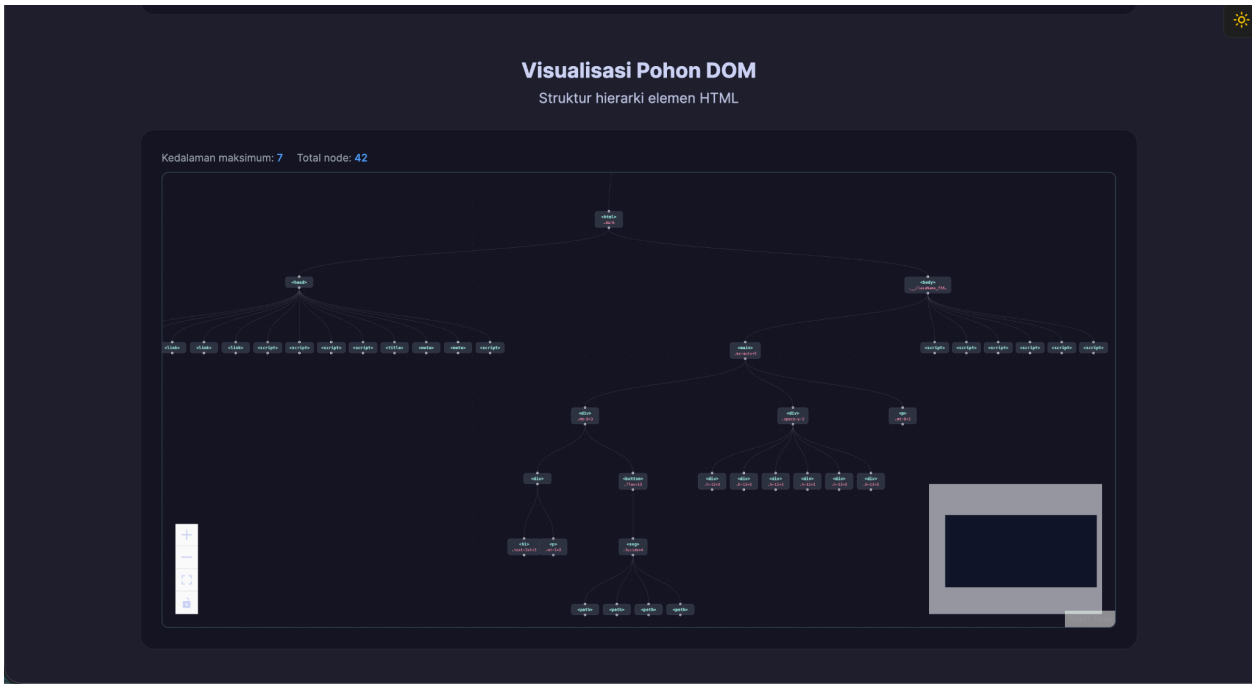
Tampilan UI

Halaman aplikasi terbagi menjadi panel input pada sisi kiri dan visualisasi pohon DOM pada sisi kanan. Hasil traversal dan traversal log ditampilkan di bawah keduanya.



Pengujian Scraping dan Pohon DOM

Scraping diuji dengan dua mode input, yaitu URL halaman web eksternal dan HTML. Hasil parsing ditampilkan dengan pohon DOM pada UI.

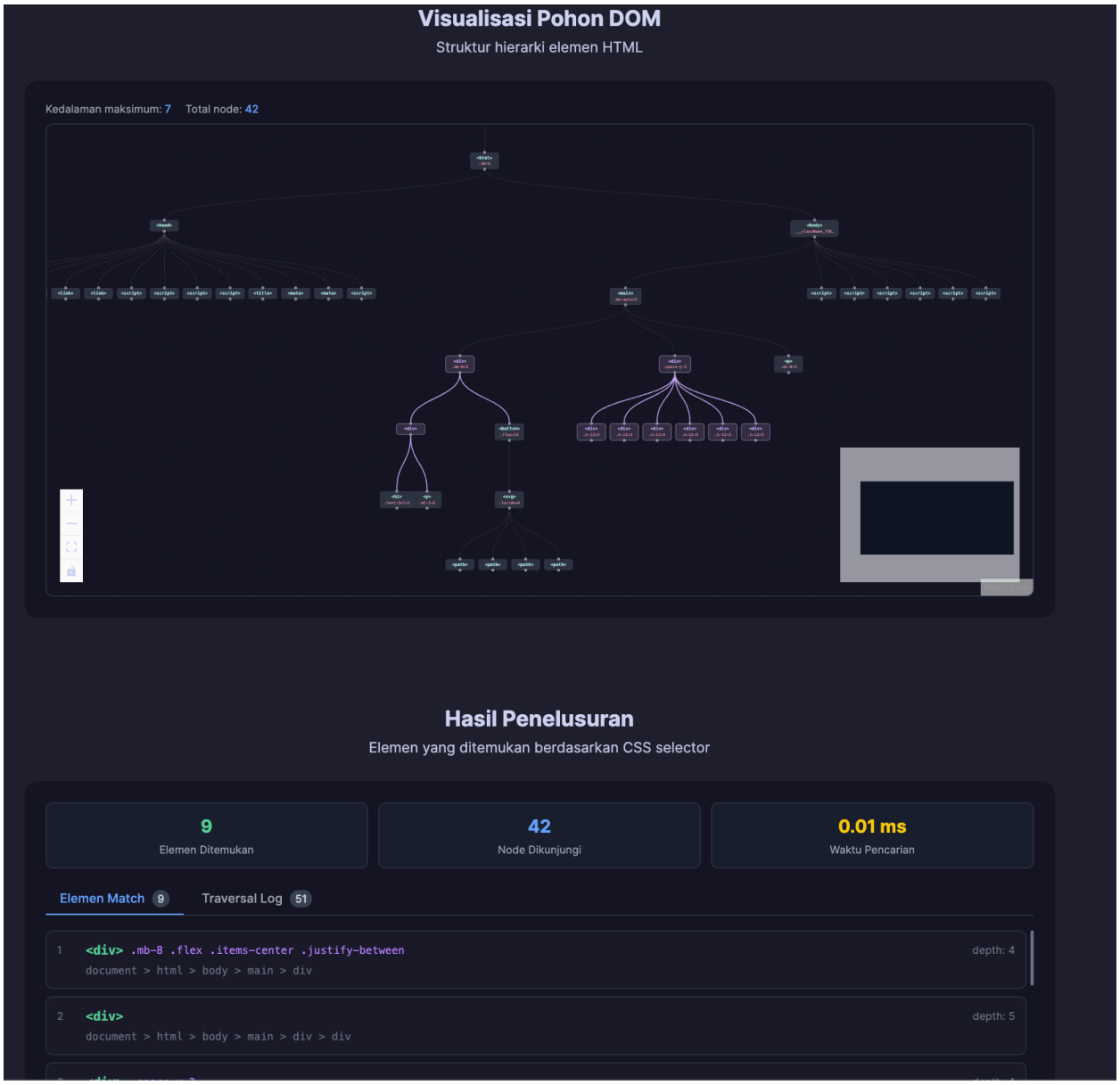


Pengujian Traversal BFS

Traversal BFS dijalankan pada pohon DOM dengan CSS Selector tertentu. Hasil berisi elemen yang ditemukan dan urutan kunjungan node yang ditampilkan pada traversal log.

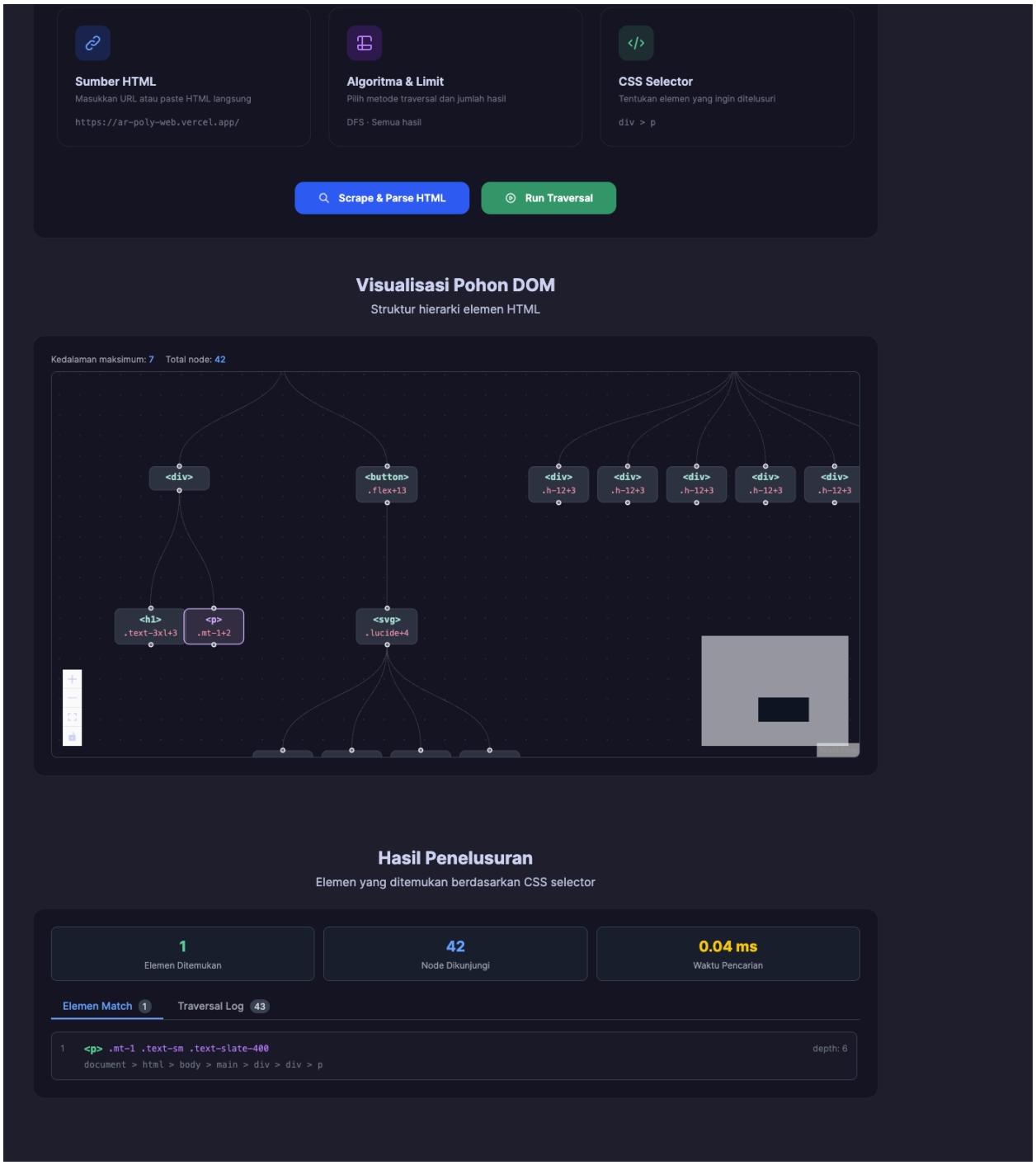
Pengujian Traversal DFS

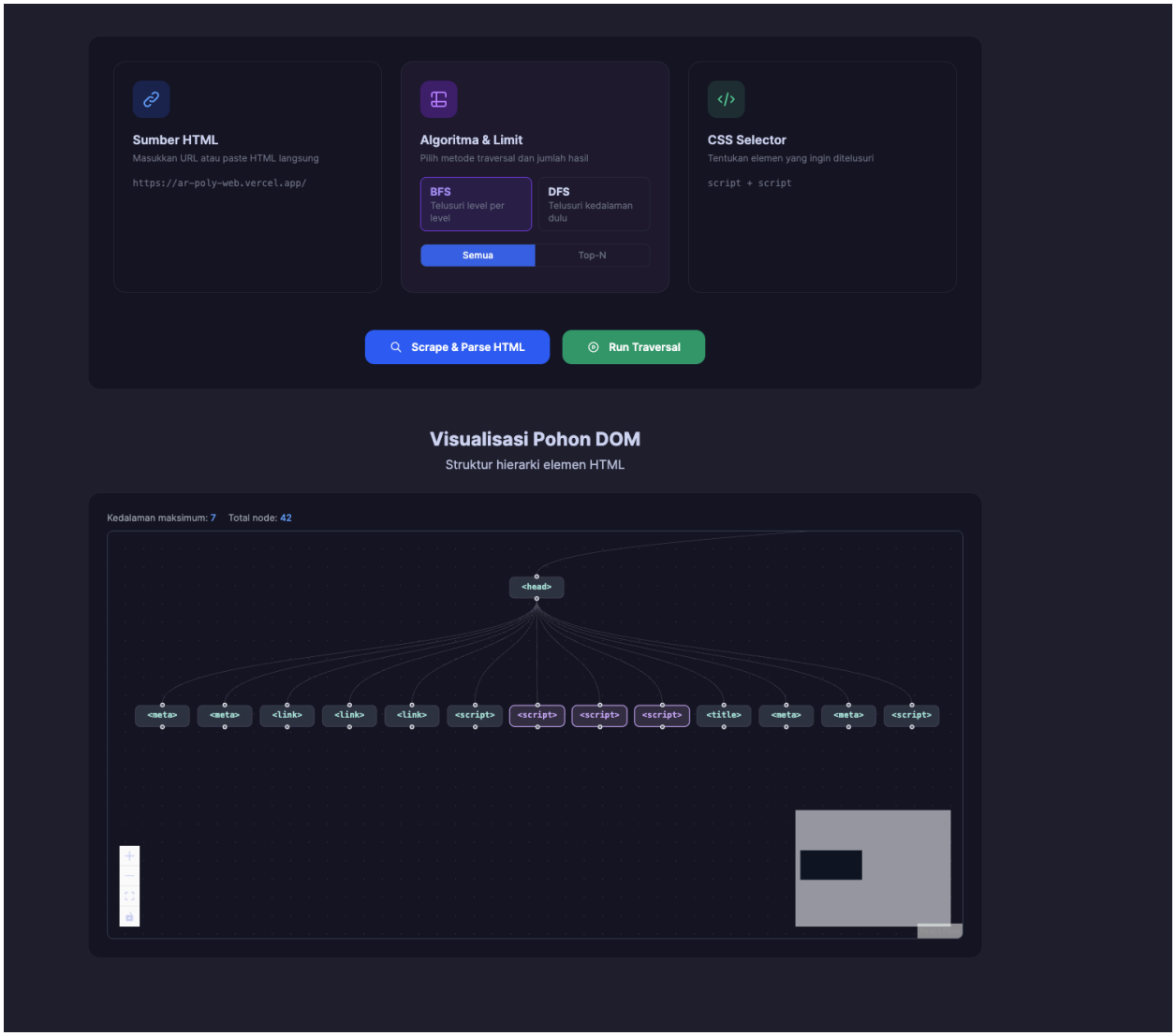
Traversal DFS dijalankan pada CSS Selector yang sama. Elemen yang ditemukan sama dengan hasil BFS, tapi urutan kunjungan node berbeda.

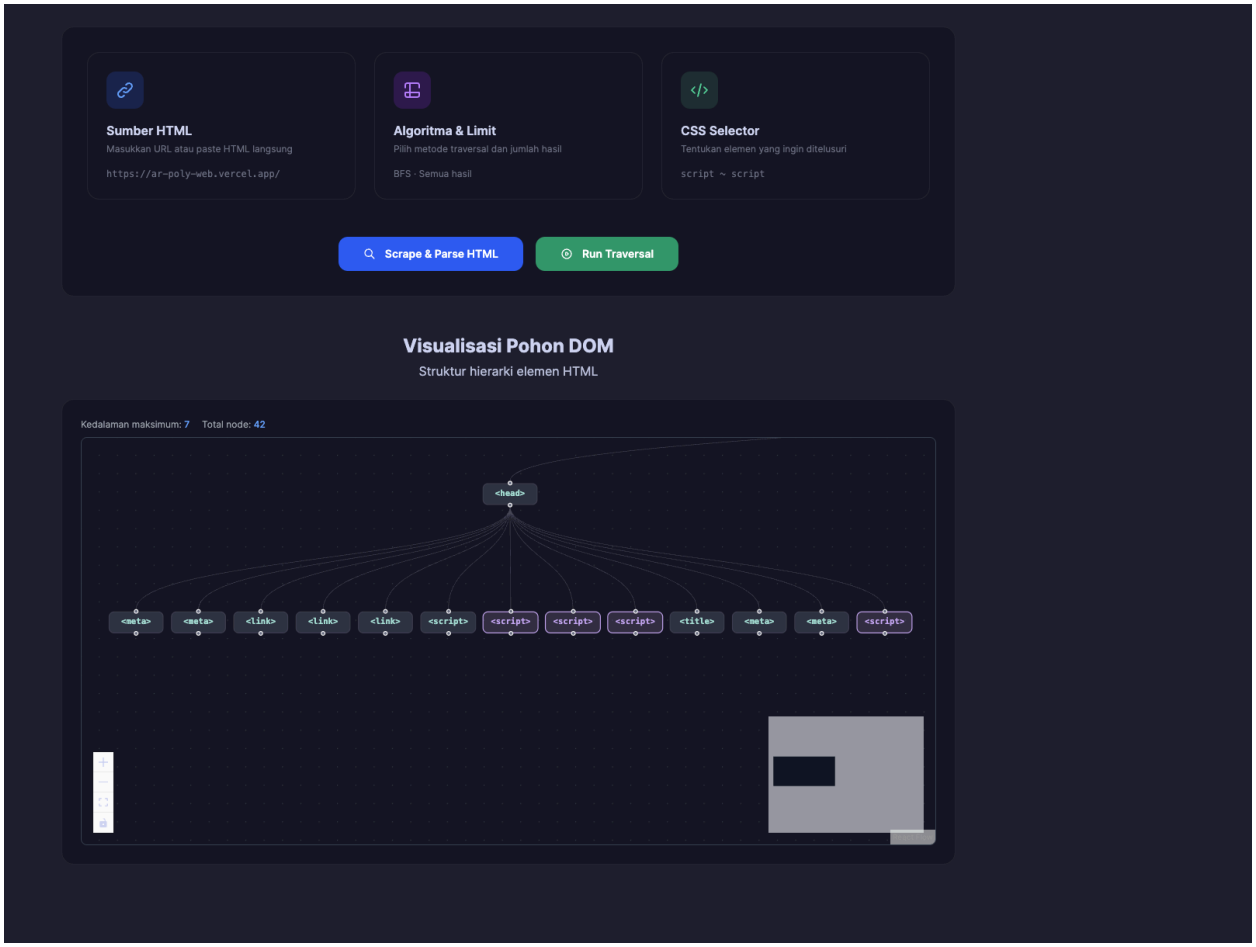


Pengujian CSS Selector

Aplikasi diuji terhadap berbagai jenis CSS Selector, meliputi tag, kelas, ID, atribut, dan kombinator descendant, child (>), adjacent sibling (+), serta general sibling (~).

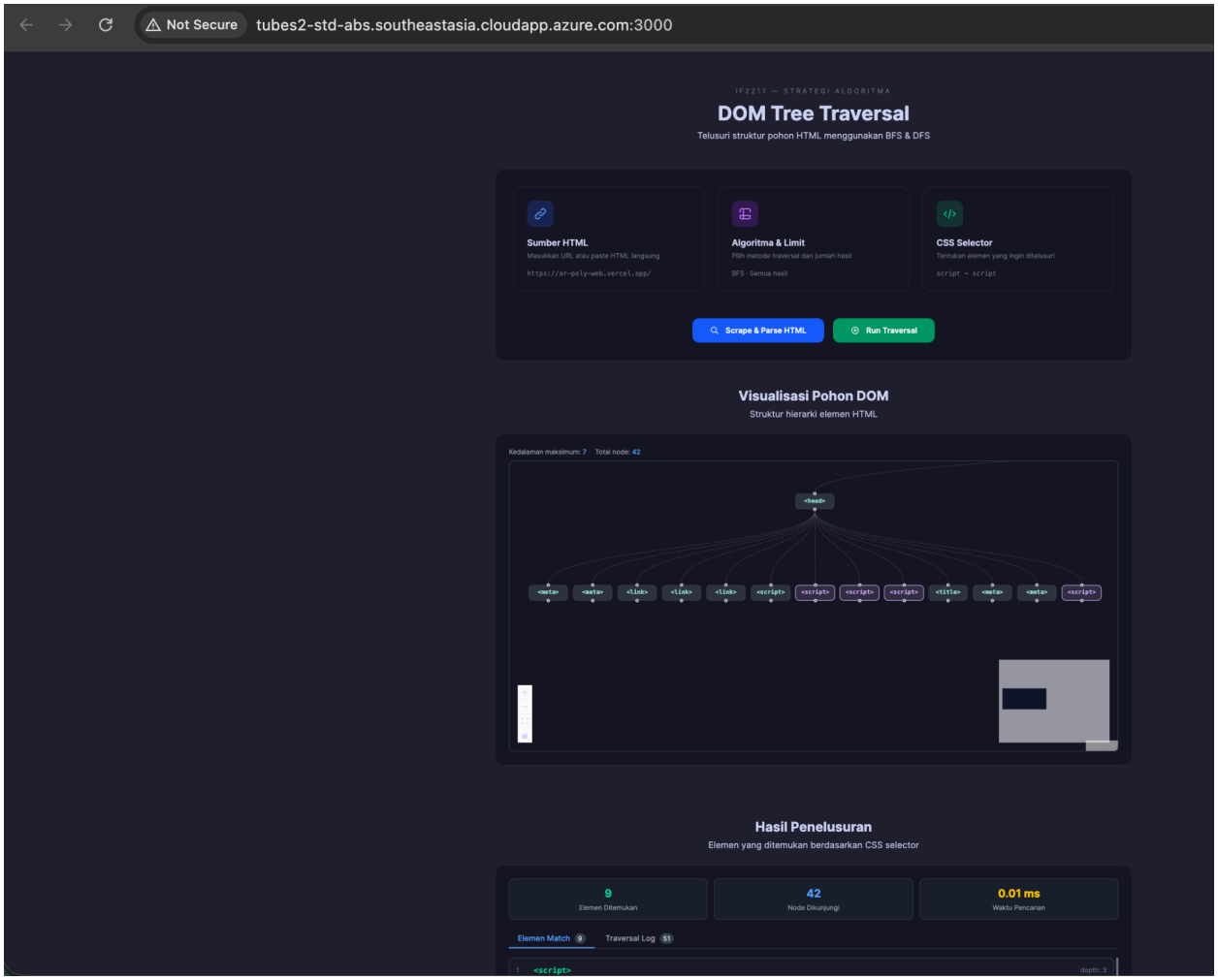



Sumber HTML
Masukkan URL atau paste HTML langsung
https://ar-poly-web.vercel.app/
Algoritma & Limit
Pilih metode traversal dan jumlah hasil
BFS
Telusuri level per level
DFS
Telusuri kedalaman dulu
Semua
Top-N
CSS Selector
Tentukan elemen yang ingin ditelusuri
script + script
Scrape & Parse HTML
Run Traversal
Visualisasi Pohon DOM
Struktur hierarki elemen HTML
Kedalaman maksimum: 7
Total node: 42
<head>
<meta>
<meta>
<link>
<link>
<link>
<script>
<script>
<script>
<script>
<title>
<meta>
<meta>
<script>


Sumber HTML
Masukkan URL atau paste HTML langsung
https://ar-poly-web.vercel.app/
Algoritma & Limit
Pilih metode traversal dan jumlah hasil
BFS · Semua hasil
CSS Selector
Tentukan elemen yang ingin ditelusuri
script ~ script
Scrape & Parse HTML
Run Traversal
Visualisasi Pohon DOM
Struktur hierarki elemen HTML
Kedalaman maksimum: 7
Total node: 42
<head>
<meta>
<meta>
<link>
<link>
<link>
<script>
<script>
<script>
<script>
<title>
<meta>
<meta>
<script>

Pengujian Deployment pada Azure VM

Aplikasi diakses melalui http://tubes2-std-abs.southeastasia.cloudapp.azure.com:3000/.




```
CONTAINER ID   IMAGE                    COMMAND                  CREATED        STATUS         PORTS
                    NAMES
fb470bf67429   tubes2_std_abs-frontend   "docker-entrypoint.s…"   10 minutes ago   Up 10 minutes   0.0.0.0:3000->3000/t
cp, [::]:3000->3000/tcp   dom_traversal_frontend
51fb477ef0a9   tubes2_std_abs-backend    "./backend"              10 minutes ago   Up 10 minutes   0.0.0.0:8080->8080/t
cp, [::]:8080->8080/tcp   dom_traversal_backend
```

BAB V KESIMPULAN DAN SARAN

Dari hasil perancangan, implementasi, dan pengujian aplikasi traversal pohon DOM menggunakan algoritma BFS dan DFS, dapat ditarik beberapa kesimpulan sebagai berikut :

1. Algoritma BFS dan DFS berhasil diimplementasikan dalam backend berbasis Rust untuk melakukan penelusuran CSS Selector pada pohon DOM yang dihasilkan dari proses parsing dokumen HTML.
2. Proses scraping HTML dan URL eksternal dan parsing menjadi struktur pohon DOM berjalan sesuai spesifikasi, menghasilkan representasi pohon yang akurat dari dokumen HTML
3. Aplikasi berhasil mendukung berbagai CSS selector meliputi tag, class, ID, universal, serta combinator child, descendant, adjacent sibling, dan general sibling.
4. BFS dan DFS menghasilkan elemen yang sama untuk CSS Selector yang sama, namun berbeda dalam urutan penemuan elemen dan karakteristik penggunaan memori. BFS menemukan elemen terdekat dari root terlebih dahulu, sedangkan DFS mengikuti urutan pre-order.

Berdasarkan pengalaman dalam mengerjakan tugas ini, terdapat beberapa saran untuk pengembangan lebih lanjut

1. Mengoptimalkan performa traversal untuk pohon DOM berukuran sangat besar, misalnya dengan caching hasil parsing atau pencocokan selector secara paralel, agar waktu eksekusi tetap rendah pada dokumen kompleks.
2. Menerapkan koneksi HTTPS pada deployment agar aplikasi dapat diakses tanpa pemblokiran web tanpa HTTPS.
3. Memperluas fitur visualisasi dengan kemampuan pencarian elemen langsung pada pohon DOM dan animasi traversal, sehingga proses penelusuran lebih mudah dipahami.

LAMPIRAN

Berikut adalah checklist kelengkapan implementasi aplikasi

NO	POIN	YA	TIDAK
1	Aplikasi berhasil dikompilasi tanpa kesalahan	✓	
2	Aplikasi berhasil dijalankan	✓	
3	Aplikasi dapat menerima input URL web, pilihan algoritma, CSS selector, dan jumlah hasil	✓	
4	Aplikasi dapat melakukan scraping terhadap web pada input	✓	
5	Aplikasi dapat menampilkan visualisasi pohon DOM	✓	
6	Aplikasi dapat menelusuri pohon DOM dan menampilkan hasil penelusuran	✓	
7	Aplikasi dapat menandai jalur tempuh oleh algoritma	✓	
8	Aplikasi dapat menyimpan jalur yang ditempuh algoritma dalam traversal log	✓	
9	[Bonus] Membuat video		✓
10	[Bonus] Deploy Aplikasi	✓	
11	[Bonus] Implementasi animasi pada penelusuran pohon		✓
12	[Bonus] Implementasi multithreading		✓
13	[Bonus] Implementasi LCA Binary Lifting		✓

Tautan Repository Github : https://github.com/Ahmad-FP/Tubes2_std_abs

Nama	Tugas
Ega Luthfi Rais	Backend (parser, handlers, scrapers, model), setup Docker, integrasi FrontEnd Backend
Ahmad Fauzan Putra	Traversal (backend), setup ms azure
Muh. Hartawan Haidir	frontend

Tugas ini disusun sepenuhnya tanpa bantuan kecerdasan buatan (*Generative AI*), melainkan hasil pemikiran dan analisis mandiri.

Ega Luthfi Rais

Ahmad Fauzan Putra

Muh. Hartawan Haidir

DAFTAR PUSTAKA

https://medium.com/@defytamara2610/bfs-breadth-first-search-pengertian-kekurangan-kelebihan-dan-contohnya-775a7d808fbc

https://www.trivusi.web.id/2022/05/apa-itu-algoritma-depth-first-search.html

https://developer.mozilla.org/en-US/docs/Web/API/Document_Object_Model

https://www.gamelab.id/news/857-berkenalan-dengan-dom-document-object-model